

Bubble Sort

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
n = int(input("Enter number of elements: "))  
arr = list(map(int, input("Enter elements: ").split()))  
  
bubble_sort(arr)  
  
print("Sorted array:", arr)
```

Selection Sort

```
import time

# Input from user
n = int(input("Enter the number of elements: "))

arr = []
print(f"Enter {n} elements:")
for i in range(n):
    arr.append(int(input()))

# Start execution time
start_time = time.perf_counter()

# Selection Sort
for i in range(n):
    min_index = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    # Swap the minimum element with the current element
    arr[i], arr[min_index] = arr[min_index], arr[i]

# End execution time
end_time = time.perf_counter()

execution_time = end_time - start_time

# Display sorted array
print("\nSorted Array:")
print(arr)

# Display execution time
print(f"\nExecution Time: {execution_time:.10f} seconds")

# Display time complexity
```

```
print("\nTime Complexity:")
print("Best Case      :  $O(n^2)$ ")
print("Average Case   :  $O(n^2)$ ")
print("Worst Case      :  $O(n^2)$ ")
print("Space Complexity:  $O(1)$ ")
```

Merge Sort

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter elements: ").split()))

merge_sort(arr)
print("Sorted array:", arr)
```

Quick Sort

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
  
    return left + middle + right  
  
n = int(input("Enter number of elements: "))  
arr = list(map(int, input("Enter elements: ").split()))  
  
sorted_arr = quick_sort(arr)  
  
print("Sorted array:", sorted_arr)
```

Linear Search

```
# Linear Search in Python

# Input the number of elements
n = int(input("Enter the number of elements: "))

# Input the array elements
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Input the element to search
key = int(input("Enter the element to search: "))

# Linear Search
found = False

for i in range(n):
    if arr[i] == key:
        print(f"Element found at position {i + 1}")
        found = True
        break

if not found:
    print("Element not found.")
```

Binary Search

```
# Binary Search in Python

# Input the number of elements
n = int(input("Enter the number of elements: "))

# Input the sorted array elements
arr = []
print("Enter the sorted elements:")
for i in range(n):
    arr.append(int(input()))

# Input the element to search
key = int(input("Enter the element to search: "))

# Binary Search
low = 0
high = n - 1
found = False

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print(f"Element found at position {mid + 1}")
        found = True
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1

if not found:
    print("Element not found.")
```